

OBJECT-ORIENTED PROGRAMMING

In modern programming, as our programs become more complex, it becomes increasingly difficult to organize code in a clear and maintainable way. Object-oriented programming (OOP) provides a solution by allowing us to structure code into small, logical units — **objects** — which have their own data and functionality. The idea is simple: we can describe the world around us through objects, and objects have **attributes** (states) and **methods** (behaviors).

Class and Object

A **class** is a kind of “blueprint” or “template” for creating objects. It defines which attributes and methods the objects of that class will have. An **object** is a concrete instance of a class. If the class is a car blueprint, the object is the actual car built according to that blueprint.

Example 5.1:

```
class Car:  
    pass # empty Class
```

This is the simplest class, which doesn't do anything yet. An object is created by calling the class:

Example 5.2:

```
my_car = Car()  
print(my_car) # <__main__.Car object at 0x...>
```

Attributes and Methods

Attributes are the data that an object stores, while **methods** are the functions that an object can perform. `__init__` is a special method that is automatically called when we create an object. It is used to initialize the object's attributes. `self` represents the object itself. Through `self`, we access the object's attributes and methods. Without `self`, Python does not know which object is being referenced.

Example 5.3:

Define the Car class

class Car:

```
    def __init__(self, brand, color):
        self.brand = brand
        # attribute: brand of the car
        self.color = color
        # attribute: color of the car

    def drive(self):
        print(f"{self.brand} is driving.")
        # drive method

    def stop(self):
        print(f"{self.brand} has stopped.")
        # stop method
```

Create an instance of the Car class

my_car = Car("Toyota", "Red") # object of Car

Access attributes and call methods

print(my_car.brand) # prints 'Toyota'

my_car.drive() # prints 'Toyota is driving.'

my_car.stop() # prints 'Toyota has stopped.'

Inheritance and Polymorphism

Inheritance allows one class to acquire the attributes and methods of another class, making code reuse easier.

Example 5.4:

```
class ElectricCar(Car):  
    def charge(self):  
        print(f"{self.brand} is charging.")  
tesla = ElectricCar("Tesla", "White")  
tesla.drive() # Tesla is driving.  
tesla.charge() # Tesla is charging.
```

Now, ElectricCar has all the methods and attributes of the Car class, plus an additional method charge().

Polymorphism means that different objects can respond differently to the same method, even though they share the same “interface” or method name.

Example 5.5:

```
class Bicycle:
    def drive(self):
        print("Bicycle is pedaling.")

vehicles = [my_car, tesla, Bicycle()]
for v in vehicles:
    v.drive()
```

Prints out:

Toyota **is** driving.
Tesla **is** driving.
Bicycle **is** pedaling.

All objects have a `drive()` method, but they behave differently.

In OOP, it is useful to think of objects as real-world entities. Each object has its own **attributes** (state) and **methods** (behavior).

PROBLEM: Create a class Dog with attributes name (dog's name) and age (years), methods bark() that prints "dog's name is barking" and celebrate_birthday() that increases the dog's age by one, then create a dog and call both methods.

SOLUTION:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def bark(self):
        print(f"{self.name} is barking.")
    def celebrate_birthday(self):
        self.age += 1 # increases age by 1
        print(f"Happy Birthday {self.name}!
        Now {self.age} years old.")

my_dog = Dog("Rex", 3)
my_dog.bark()
my_dog.celebrate_birthday()
```
